جزوه جامع و بی‌نظیر سیستم عامل (بخش ۶ تا ۹: مفاهیم پیشرفته)

این جزوه بر اساس بخش‌های ششم تا نهم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی، فرآیندها و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۶-۱: امنیت و حفاظت (Security and Protection)

در یک سیستم چندکاربره و چندوظیفه‌ای، حفاظت از منابع و تضمین امنیت، از حیاتی‌ترین وظایف سیستم‌عامل است. این دو مفهوم نزدیک به هم هستند اما تفاوت‌های مهمی دارند.

- **حفاظت (Protection):**
  - **تعریف:** حفاظت به مکانیزم‌های **داخلی** سیستم‌عامل برای **کنترل دسترسی** پردازه‌ها یا کاربران به منابع تعریف شده در سیستم گفته می‌شود. هدف آن، جلوگیری از تداخل ناخواسته یا مخرب بین اجزای خود سیستم است.
  - **مثال‌ها:**
    - **حفاظت حافظه:** سخت‌افزار آدرس‌دهی حافظه تضمین می‌کند که یک پردازه فقط می‌تواند به فضای آدرس خودش دسترسی داشته باشد و نمی‌تواند حافظه پردازه‌های دیگر یا خود سیستم‌عامل را بخواند یا تغییر دهد.
    - **حفاظت CPU:** تایمر تضمین می‌کند که هیچ پردازه‌ای نمی‌تواند CPU را برای همیشه در انحصار خود بگیرد.
    - **حفاظت I/O:** دستورالعمل‌های مربوط به I/O ممتاز هستند و تنها از طریق سیستم‌عامل قابل دسترسی‌اند، بنابراین برنامه‌های کاربر نمی‌توانند مستقیماً با سخت‌افزار کار کنند.
  - **هدف:** حفاظت، یک سیستم قابل اعتماد می‌سازد و از گسترش خطا از یک زیرسیستم به زیرسیستم دیگر جلوگیری می‌کند.
- **امنیت (Security):**
  - **تعریف:** امنیت به دفاع از سیستم در برابر حملات **داخلی و خارجی** می‌پردازد. هدف آن، جلوگیری از دسترسی یا استفاده غیرمجاز از سیستم است.
  - **مثال‌های تهدیدات امنیتی:**
    - **ویروس‌ها و کرم‌ها (Viruses and Worms):** برنامه‌های مخربی که خود را تکثیر کرده و به سیستم آسیب می‌رسانند.
    - **ح حملات ممانعت از سرویس (Denial of Service - DoS):** حملاتی که با ایجاد بار کاری سنگین و بی‌رویه، منابع سیستم را اشغال کرده و مانع از سرویس‌دهی به کاربران قانونی می‌شوند.
    - **سرقت شناسه (Identity Theft) و سرقت سرویس (Theft of Service):** استفاده غیرمجاز از هویت یک کاربر دیگر یا منابع سیستم.
  - **تفاوت کلیدی:** حفاظت، مکانیزم‌های لازم برای اعمال سیاست‌های امنیتی را فراهم می‌کند. امنیت، خود آن سیاست‌هاست. برای مثال، مکانیزم حفاظت فایل (تعیین مجوزهای خواندن/نوشتن) وجود دارد؛ سیاست امنیتی می‌گوید "کاربر A حق نوشتن در فایل B را ندارد".

- **مکانیزم‌های شناسایی کاربر:** برای اعمال حفاظت و امنیت، سیستم‌عامل باید بتواند کاربران را از یکدیگر تمییز دهد.
  - **شناسه کاربری (User ID - UID):** به هر کاربر یک شناسه عددی منحصربه‌فرد اختصاص داده می‌شود. در ویندوز به آن **شناسه امنیتی (Security ID - SID)** می‌گویند. تمام پردازه‌هایی که توسط یک کاربر اجرا می‌شوند، با UID او برچسب‌گذاری می‌شوند.
  - **شناسه گروهی (Group ID - GID):** برای مدیریت دسترسی گروهی از کاربران، از GID استفاده می‌شود. یک کاربر می‌تواند عضو یک یا چند گروه باشد.
  - **ارتقای امتیاز (Privilege Escalation):** گاهی یک کاربر نیاز دارد تا برای انجام یک کار خاص، به طور موقت امتیازات بیشتری کسب کند. برای مثال، در یونیکس، با استفاده از صفت `setuid` روی یک برنامه، آن برنامه با UID مالک فایل (که معمولاً کاربر ریشه یا root است) اجرا می‌شود، نه با UID کاربری که آن را اجرا کرده است.

۷-۱: مجازی‌سازی (Virtualization)

مجازی‌سازی یک فناوری قدرتمند است که به ما اجازه می‌دهد سخت‌افزار یک کامپیوتر فیزیکی را به چندین **محیط اجرایی مجازی** تقسیم کنیم. هر یک از این محیط‌ها، که به آن‌ها **ماشین مجازی (Virtual Machine - VM)** گفته می‌شود، مانند یک کامپیوتر کاملاً مستقل عمل می‌کند.

- **ایده کلیدی:** یک لایه نرم‌افزاری به نام **مدیر ماشین مجازی (Virtual Machine Manager - VMM)** یا **Hypervisor**، مستقیماً روی سخت‌افزار (یا روی یک سیستم‌عامل میزبان) قرار می‌گیرد. VMM سخت‌افزار فیزیکی (CPU، حافظه، دیسک) را به چندین نسخه مجازی تقسیم کرده و به هر VM اختصاص می‌دهد. سپس، می‌توان روی هر VM یک سیستم‌عامل کامل و مجزا (که به آن **سیستم‌عامل میهمان (Guest OS)** گفته می‌شود) نصب و اجرا کرد.
- **تفاوت با تقلید (Emulation):**
  - **تقلید:** زمانی استفاده می‌شود که معماری پردازنده مبدأ با مقصد **متفاوت** است (مثلاً اجرای یک برنامه نوشته شده برای پردازنده PowerPC روی یک پردازنده Intel). در این حالت، هر دستورالعمل مبدأ باید به یک یا چند دستورالعمل مقصد ترجمه شود که این فرآیند بسیار **کند** است.
  - **مجازی‌سازی:** در اینجا، معماری پردازنده میهمان و میزبان **یکسان** است. VMM بیشتر دستورالعمل‌ها را مستقیماً روی سخت‌افزار اجرا می‌کند و تنها دستورالعمل‌های حساس و ممتاز را مدیریت می‌کند، به همین دلیل کارایی آن بسیار بالاتر از تقلید است.
- **کاربردها:**
  - **مراکز داده (Data Centers):** برای تجمیع چندین سرور روی یک سخت‌افزار فیزیکی واحد، که باعث صرفه‌جویی در هزینه، انرژی و فضا می‌شود.
  - **توسعه و تست نرم‌افزار:** برنامه‌نویسان می‌توانند برنامه‌های خود را به راحتی روی چندین سیستم‌عامل مختلف (مثلاً ویندوز، لینوکس و مک) روی یک کامپیوتر تست کنند.
  - **اجرای برنامه‌های قدیمی:** اجرای برنامه‌هایی که فقط روی نسخه‌های قدیمی‌تر یک سیستم‌عامل کار می‌کنند.
  - **امنیت:** ایزوله کردن برنامه‌ها در یک VM برای جلوگیری از آسیب رساندن به سیستم اصلی.

۱-۸: سیستم‌های توزیع‌شده (Distributed Systems)

یک سیستم توزیع‌شده، مجموعه‌ای از کامپیوترهای فیزیکی مجزا است که از طریق یک **شبکه (Network)** به هم متصل شده‌اند و با یکدیگر همکاری می‌کنند تا یک هدف مشترک را دنبال کنند. برای کاربر نهایی، این مجموعه ممکن است مانند یک سیستم واحد و یکپارچه به نظر برسد.

- **مزایا:**
  - **اشتراک منابع (Resource Sharing):** دسترسی به فایل‌ها، پرینترها و سایر منابع در سراسر شبکه.
  - **افزایش سرعت محاسبات (Computation Speedup):** تقسیم یک کار بزرگ بین چندین کامپیوتر و اجرای موازی آن‌ها.
  - **افزایش قابلیت اطمینان (Reliability):** در صورت خرابی یک کامپیوتر، کامپیوترهای دیگر می‌توانند به کار خود ادامه دهند.
  - **ارتباطات (Communication):** تبادل اطلاعات بین کاربران و پردازه‌ها.
- **شبکه (Network):**شبکه، مسیر ارتباطی بین سیستم‌هاست. شبکه‌ها بر اساس **پروتکل** (مانند TCP/IP که پایه اینترنت است)، **فاصله** و **رسانه انتقال** دسته‌بندی می‌شوند.
  - **انواع شبکه بر اساس فاصله:**
    - **LAN (Local-Area Network):** شبکه محلی که کامپیوترها را در یک ساختمان یا پردیس به هم متصل می‌کند.
    - **WAN (Wide-Area Network):** شبکه گسترده که شهرها یا کشورها را به هم متصل می‌کند.
    - **MAN (Metropolitan-Area Network):** شبکه شهری.
    - **PAN (Personal-Area Network):** شبکه شخصی در فاصله چند متری (مانند بلوتوث).
- **سیستم‌عامل شبکه در مقابل سیستم‌عامل توزیع‌شده:**
  - **سیستم‌عامل شبکه (Network OS):** هر کامپیوتر سیستم‌عامل مستقل خود را دارد اما از وجود شبکه آگاه است و می‌تواند با دیگران ارتباط برقرار کرده و منابع را به اشتراک بگذارد (مانند اشتراک فایل در ویندوز).
  - **سیستم‌عامل توزیع‌شده (Distributed OS):** یک لایه نرم‌افزاری یکپارچه روی تمام کامپیوترها اجرا می‌شود و کل مجموعه را مانند یک کامپیوتر واحد مدیریت می‌کند. این نوع سیستم‌عامل استقلال کمتری به گره‌ها می‌دهد.

۱-۹: ساختمان داده‌های هسته (Kernel Data Structures)

هسته سیستم‌عامل برای مدیریت کارآمد و سریع اطلاعات مربوط به پردازه‌ها، حافظه، فایل‌ها و سایر منابع، از ساختمان داده‌های کلاسیک علوم کامپیوتر استفاده می‌کند. انتخاب ساختمان داده مناسب، تأثیر مستقیمی بر کارایی سیستم‌عامل دارد.

- **لیست‌های پیوندی (Linked Lists):**برای مدیریت مجموعه‌هایی با اندازه متغیر بسیار پرکاربرد هستند.
  - **یک‌طرفه:** هر عنصر فقط به عنصر بعدی اشاره می‌کند.
  - **دوطرفه:** هر عنصر به عناصر قبلی و بعدی اشاره می‌کند.
  - **چرخشی:** آخرین عنصر به اولین عنصر اشاره می‌کند.
- **پشته (Stack):**
  - **ساختار:** با منطق **LIFO** (آخرین ورود، اولین **خروج**) کار می‌کند.
  - **کاربرد:** برای مدیریت **فراخوانی توابع** استفاده می‌شود. پارامترها، متغیرهای محلی و آدرس بازگشت تابع در پشته قرار گرفته (Push) و پس از بازگشت از تابع، از پشته برداشته می‌شوند (Pop).

- **صف (Queue):**
  - **ساختار:** با منطق **FIFO (اولین ورود، اولین خروج)** کار می‌کند.
  - **کاربرد:** برای مدیریت وظایف در صف انتظار استفاده می‌شود. مانند لیست پردازه‌های منتظر CPU، یا لیست کارهای ارسالی به چاپگر.
- **درخت (Tree):**برای نمایش داده‌های سلسله مراتبی و جستجوی سریع استفاده می‌شود.
  - **درخت جستجوی دودویی (Binary Search Tree):** برای هر گره، تمام مقادیر در زیردرخت چپ، کوچکتر و تمام مقادیر در زیردرخت راست، بزرگتر هستند.
  - **درخت جستجوی دودویی متوازن:** برای جلوگیری از بدترین حالت (که شبیه لیست پیوندی می‌شود)، از الگوریتم‌هایی برای متوازن نگه داشتن درخت استفاده می‌شود. برای مثال، سیستم‌عامل **لینوکس** از یک **درخت قرمز-سیاه (Red-Black Tree)** در الگوریتم زمان‌بندی پردازنده خود برای مدیریت کارآمد پردازه‌ها استفاده می‌کند.
- **نقشه درهم (Hash Map) و تابع درهم (Hash Function):**
  - **تابع درهم:** یک تابع ریاضی که یک ورودی با اندازه متغیر را به یک خروجی عددی با اندازه ثابت (اندیس جدول) نگاشت می‌کند.
  - **نقشه درهم:** از تابع درهم برای ذخیره و بازیابی زوج‌های **کلید-مقدار (Key-Value)** با سرعت بسیار بالا (در حالت ایده‌آل با زمان ثابت O(1)) استفاده می‌کند.
  - **مشکل برخورد (Collision):** اگر دو کلید مختلف به یک اندیس یکسان نگاشت شوند، از روش‌هایی مانند **زنجیره‌سازی (Chaining)** (استفاده از لیست پیوندی در آن خانه از جدول) برای حل مشکل استفاده می‌شود.
- **نقشه بیتی (Bitmap):**
  - **ساختار:** یک رشته فشرده از بیت‌ها که برای نمایش وضعیت مجموعه‌ای از منابع استفاده می‌شود.
  - **کاربرد:** بسیار بهینه از نظر فضا. برای مثال، برای نمایش وضعیت بلوک‌های آزاد و اشغال شده روی یک دیسک سخت، می‌توان از یک نقشه بیتی استفاده کرد که در